CAA Lab Assignment 6

```
struct CPU {
  unsigned char R[4];
  unsigned char ACC;
  unsigned char PC;
  unsigned char Z, C;
  unsigned char EPC;
  unsigned char CAUSE;
  unsigned char IE;
};

struct IFEX {
  unsigned char IR;
  unsigned char PC;
  int valid;
  int predicted_taken;
};

struct BPEntry {
  int valid;
  int counter;
};

void reset_cpu() {
  memset(&cpu, 0, sizeof(cpu));
  memset(&ifex, 0, sizeof(ifex));
  memset(bp_table, 0, sizeof(bp_table));
  cpu.IE = 1;
  cpu.R[1] = 1;
  cpu.R[2] = 0;
}

void load_program(const char* file) {
  ifstream fin(file);
  if (!fin) {
    cout << "Program load error\n";
    exit(1);
  }
  int i = 0;
```

```
  unsigned int x;
  while (fin >> hex >> x)
    IMEM[i++] = (unsigned char)x;
  fin.close();
}

int bp_index(unsigned char pc) {
  return pc % BP_SIZE;
}

int predict_branch(unsigned char pc) {
  int idx = bp_index(pc);
  if (bp_table[idx].valid)
    return bp_table[idx].counter >= 2;
  return 0;
}

void update_predictor(unsigned char pc, int taken) {
  int idx = bp_index(pc);
  bp_table[idx].valid = 1;

  if (taken && bp_table[idx].counter < 3)
    bp_table[idx].counter++;
  else if (!taken && bp_table[idx].counter > 0)
    bp_table[idx].counter--;
}

uint8_t alu(uint8_t a, uint8_t b, int op) {
  int res = 0;
  if (op == 0x4) res = a + b;
  if (op == 0x5) res = a - b;
  if (op == 0x6) res = a * b;
  if (op == 0x7) res = (b == 0 ? 0 : a / b);
  cpu.Z = (res == 0);
  cpu.C = (res > 255);
  return res & 0xFF;
}

int interrupt_pending() {
  return (cpu.IE && (cycles % TIMER_PERIOD == 0));
}

void raise_interrupt(int cause) {
```

```
  cpu.EPC = cpu.PC;
  cpu.CAUSE = cause;
  cpu.IE = 0;
  ifex.valid = 0;
  cpu.PC = INT_HANDLER;
  interrupts++;
}

void raise_exception(int cause) {
  cpu.EPC = cpu.PC;
  cpu.CAUSE = cause;
  cpu.IE = 0;
  ifex.valid = 0;
  cpu.PC = INT_HANDLER;
  exceptions++;
}

void fetch_stage() {
  ifex.IR = IMEM[cpu.PC++];
  ifex.PC = cpu.PC;
  ifex.valid = 1;
  unsigned char op = (ifex.IR >> 4) & 0xF;
  if (op == OP_JMP || op == OP_JZ) {
    int pred = predict_branch(cpu.PC);
    ifex.predicted_taken = pred;
    unsigned char operand = ifex.IR & 0xF;
    cpu.PC = pred ? operand : cpu.PC + 1;
  }
}

void execute_stage() {
  if (!ifex.valid) return;
  unsigned char ir = ifex.IR;
  unsigned char op = (ir >> 4) & 0xF;
  unsigned char operand = ir & 0xF;
  unsigned char arg = ir & 0xF;
  instructions++;

  switch (op) {
  case OP_MOVI:
    cpu.ACC = arg;
    break;
  case OP_LOAD:
```

```
    cpu.ACC = cpu.R[arg];
    break;
  case OP_STORE:
    cpu.R[arg] = cpu.ACC;
    break;
  case OP_ADD:
    cpu.ACC += cpu.R[arg];
    break;
  case OP_SUB:
    cpu.ACC -= cpu.R[arg];
    break;
  case OP_DIV:
    if (cpu.R[arg] == 0) {
      raise_exception(1);
      return;
    }
    cpu.ACC /= cpu.R[arg];
    break;
  case OP_CMP:
    cpu.Z = (cpu.ACC == cpu.R[arg]);
    break;
  case OP_JMP:
    cpu.PC = operand;
    jump_taken = true;
    break;
  }
}
```